Загрузка файлов

- Создание формы загрузки
- Создание обработчика формы
- Перемещение загруженного файла
- Мультизагрузка
- Выбор имени для файла

Создание формы загрузки

Кроме текстовой информации, существует возможность отправлять на сервер файлы **любых типов**. Ниже представлен пример **формы** для загрузки файла.

example_1/form.html. Форма загрузки файла

```
<h1>Отправка файлов на сервер</h1>

<form method="post" action="server.php" enctype="multipart/form-data">

     Логин: <input type="text" name="login"><p>

     <input type="hidden" name="MAX_FILE_SIZE" value="150000"><p>

     Выбор файла для загрузки: <input type="file"
     name="myfile"><p>

     <input type="submit" name="load" value="Загрузить файл">

</form>
```

При создании формы загрузки **обратите внимание** на следующие моменты:

- Атрибут **method** установлен в значение **post**.
- Присутствует атрибут **enctype**, который всегда должен иметь значение **multipart/form-data**. Если его не будет, то **файл не отправится**.

- Форма содержит специальное **поле для выбора файла**, элемент формы типа **file**: <input **type="file"** ...>
- В форме **опционально** может находиться скрытое поле с атрибутом **name**=MAX_FILE_SIZE указывающее на максимальный размер в байтах загружаемых файлов. Особое внимание нужно обратить на то, что этот элемент должен стоять **до** поля / полей выбора файлов.

> ***К сведению***. Поле **MAX_FILE_SIZE** следует воспринимать как пожелание разработчика о размере файла. Поле должно предшествовать полю для выбора файла, и его значение является **максимально допустимым размером** принимаемого файла в PHP.
>
> Поддерживается не всеми браузерами.

В открытом доступе, при желании, найдете достаточно дискуссий о целесообразности применения данного поля.

Все загружаемые файлы попадают в **суперглобальный ассоциативный массив $_FILES**. Чтобы просмотреть информацию о загруженном файле, массив $_FILES как любой другой, можно вывести с помощью функции **var_dump**:

```
<?php
    echo "<pre>";
    var_dump($_FILES);
    echo "</pre>";
```

> ***Таким образом***. Чтобы загрузить файл на сервер, необходимо:
>
> - использовать **POST-форму**
> - параметр **enctype** формы установить в **multipart/form-data**
> - на сервере использовать массив **$_FILES**.

Создание обработчика формы

Простой обработчик формы

Ассоциативный массив **$_FILES является двумерным**. Доступ к загружаемому файлу можно получить по ключу, который определяется значением атрибута **name** элемента выбора файла с **type="file"**:

```
<input type="file" name="myfile">
```

Так как элемент для выбора файла из примера **example_1** имеет name="myfile", то загружаемый файл мы можем получить с помощью массива **$_FILES**[**"myfile"**].

У каждого объекта загружаемого файла есть общедоступные параметры:

- **$_FILES**[**"myfile"**][**"name"**] - имя файла.
- **$_FILES**[**"myfile"**][**"type"**] - тип содержимого файла, например, image/jpeg.
- **$_FILES**[**"myfile"**][**"size"**] - размер файла в байтах.
- **$_FILES**[**"myfile"**][**"tmp_name"**] – временное имя файла, сохраненного на сервере.
- **$_FILES**[**"myfile"**][**"error"**] - код ошибки при загрузке. Если у нас нет ошибки, то поле содержит значение **UPLOAD_ERR_OK**. Неполный список констант ошибок приведен в таблице:

Константа	Описание
UPLOAD_ERR_OK	Значение: 0; Файл был успешно загружен на сервер
UPLOAD_ERR_INI_SIZE	Значение: 1; Размер принятого файла превысил максимально допустимый размер, который задан директивой **upload_max_filesize** конфигурационного файла **php.ini**.

UPLOAD_ERR_FORM_SIZE	Значение: 2; Размер загружаемого файла превысил значение **MAX_FILE_SIZE**, указанное в HTML-форме.
UPLOAD_ERR_PARTIAL	Значение: 3; Загружаемый файл был получен только частично.
UPLOAD_ERR_NO_FILE	Значение: 4; Файл не был загружен.

* - полный список констант можно посмотреть в открытых источниках

В листинге **example_1/server.php** представлен код обработчика формы загрузки файла. Создайте обработчик, протестируйте форму загрузки.

> ***Важно***. В форме выставлен критерий максимального размера загружаемого файла. Будьте внимательны. При необходимости планку критерия можно **поднять** или вообще **убрать** поле **name="MAX_FILE_SIZE"**.

Обратите внимание, с принятым в обработчике файлом мы пока ничего не делаем, просто фиксируем факт загрузки и выводим информацию о файле из массива $_**FILES**.

example_1/server.php. Создание обработчика формы

```
<?php
    // доступ к файлу осуществляется через двумерный массив
    $_FILES
    // проверим, пытается ли пользователь загрузить файл
    if (isset($_FILES['myfile'])) {
        echo "<h2>На сервере принят файл:</h2>";
        // выводим информацию о загружаемом файле
        printf ("
            <i>Имя файла:</i> <b> %s </b>, <p>
```

```
                <i>Размер файла:</i> <b> %s </b> байт, <p>
                <i>Тип содержимого файла:</i> <b> %s </b>, <p>
                <i>Имя временного файла:</i> <b> %s </b>, <p>
                <i>Код ошибки:</i> <b> %s </b>
                ",
                $_FILES['myfile']['name'],
                $_FILES['myfile']['size'],
                $_FILES["myfile"]["type"],
                $_FILES["myfile"]["tmp_name"],
                $_FILES["myfile"]["error"]
            );
            // файл был загружен во временную директорию
        } else {
            echo "<h3>Заполните, пожалуйста форму!</h3>";
        }
        // проблема в том, что ...
        // -> файлы временной директории очищаются после окончания
        работы скрипта
?>
```

> ***Важно.*** Отправленный пользователем файл **принят** сервером. Но **сохранением** файла займемся дальше. Дело в том, что файлы временной директории **очищаются после окончания работы скрипта**.

В скрипте есть один небольшой нюанс. Вы должны хорошо представлять себе смысл следующей строчки кода:

```
// проверим, пытается ли пользователь загрузить файл
```

```
if (isset($_FILES['myfile'])) {
// ...
```

Условие будет выполнено, вне зависимости от того, пытаемся мы загрузить файл или файл для загрузки выбран не был. Чтобы ответить на вопрос, зачем здесь эта проверка, попробуйте перейти в **скрипт обработчика формы** по прямой ссылке. У меня эта ссылка выглядит так:

http://php-course/**example_1/example_1_2.php**.

Скрипт должен выполняться **только как обработчик формы загрузки файла**, и обойти этап заполнения формы нельзя.

Обработчик формы с проверкой ошибок

При кодинге обработчика из листинга example_1_2, мы **вывели** информацию о возможных ошибках загрузки, но никак эту информацию не использовали. Усовершенствуем обработчик формы и проанализируем массив $_FILES["myfile"]["error"] на предмет появления в нем ошибок.

> ***Важно***. "Мы не можем ждать милостей от природы. Взять их у нее - наша задача". Это в том смысле, что мы не можем ждать появления ошибок, вызвать их – наша задача.
>
> Например, легко вызвать ошибку, намеренно уменьшив значение поля **MAX_FILE_SIZE** формы, или нажав кнопку **Отправить,** не выбрав файл.

example_2/form.html. Форма загрузки файла

```
<h1>Отправка файлов на сервер</h1>
<form method="post" action="server.php" enctype="multipart/form-data">
```

```
    Логин: <input type="text" name="login"><p>
    <!-- следите за значением скрытого поля MAX_FILE_SIZE -->
    <input type="hidden" name="MAX_FILE_SIZE" value="1000"><p>
    Выбор файла для загрузки: <input type="file"
    name="myfile"><p>
    <input type="submit" name="load" value="Загрузить файл">
</form>
```

Будьте внимательны при анализе и наборе кода обработчика. Скрипт получился не маленьким, но сложным его, точно, назвать нельзя.

example_2/server.php. Проверка кода ошибки в обработчике формы

```
<?php
    // доступ к файлу осуществляется через двумерный массив
    $_FILES
    // проверим загрузку на наличие ошибок
    if ($_FILES['myfile']["error"] == UPLOAD_ERR_OK) {
        // если файл загружен выводим информацию о нем
        echo "<h2>На сервере принят файл:</h2>";
        printf ("
            <i>Имя файла:</i> <b> %s </b>, <p>
            <i>Размер файла:</i> <b> %s </b> байт, <p>
            <i>Тип содержимого файла:</i> <b> %s </b>, <p>
            <i>Имя временного файла:</i> <b> %s </b>, <p>
            <i>Код ошибки:</i> <b> %s </b>
            ",
            $_FILES['myfile']['name'],
            $_FILES['myfile']['size'],
            $_FILES["myfile"]["type"],
```

```
            $_FILES["myfile"]["tmp_name"],
            $_FILES["myfile"]["error"]
        );
        // файл загружен во временную директорию
    } else {
        // если при загрузке произошла ошибка, выводим
        информацию о ней
        switch ($_FILES['myfile']['error']) {
            case UPLOAD_ERR_INI_SIZE:
            echo "<h3>Размер принятого файла превысил
            максимально допустимый размер, который задан
            директивой upload_max_filesize конфигурационного
            файла php.ini (код ошибки: 1)</h3>";
            break;
            case UPLOAD_ERR_FORM_SIZE:
            echo "<h3>Размер загружаемого файла превысил
            значение MAX_FILE_SIZE, указанное в HTML-форме
            (код ошибки: 2)</h3>";
            break;
            case UPLOAD_ERR_PARTIAL:
            echo "<h3>Загружаемый файл был получен только
            частично (код ошибки: 3)</h3>";
            break;
            case UPLOAD_ERR_NO_FILE:
            echo "<h3>Файл не был загружен (код ошибки:
            4)</h3>";
        }
    }
    // проблема в том, что ...
```

```
    // -> файлы временной директории очищаются после работы
    скрипта
?>
```

PHP автоматически сохраняет все загруженные файлы во **временной директории сервера**. Но хранить там файлы нельзя, потому что эта директория автоматически очищается, и ссылку на такой файл хранить бессмысленно.

Решение здесь только одно — переместить загруженный файл в другую директорию. **Перемещение файла** в директорию сервера для постоянного хранения **всегда выполняют сразу после загрузки** с помощью функции **move_uploaded_file**().

Перемещение загруженного файла

Для тестирования возможности перемещения загружаемого файла в директорию постоянного хранения сервера создадим форму выбора **аватара**.

example_3/form.html. Форма загрузки файла

```
<h1>Отправка файлов на сервер</h1>
<form action="server.php" enctype="multipart/form-data"
method="post">
    Загрузите ваш аватар: <input type="file" name="avatar">
    <input type="submit" name="load" value="Загрузить файл">
</form>
```

Перед началом перемещения нужно убедиться, что в рабочем каталоге проекта существует директория для **хранения загруженных файлов**. Пусть в нашем случае такая директория называется **upload**.

> ***Важно.*** Проверьте, что в директории размещения файла сценария у вас создан каталог **upload**.

Для перемещения файла необходимо знать, **текущее расположение файла**, и **место постоянного хранения файла** на сервере.

С текущим адресом всё крайне просто — файл находится в массиве $_FILES. За текущее расположение файла отвечает параметр: **$_FILES**["**myfile**"]["**tmp_name**"].

```
<?php
    // текущее расположение файла
    $current_path = $_FILES['avatar']['tmp_name'];
```

Новый адрес файла, в свою очередь, состоит из **пути к директории постоянного хранения** и непосредственно **имени файла**. Так как директория **upload** находится там же, где и текущий сценарий, получить путь к ней можно с помощью магической константы: **__DIR__**.

```
<?php
    // оригинальное имя файла
    $filename = $_FILES['avatar']['name'];
    // место постоянного хранения файла
    $new_path = __DIR__ . '/upload/' . $filename;
```

Текущее расположение загружаемого файла известно, постоянное место хранения файла известно. Осталось переместить файл.

move_uploaded_file

move_uploaded_file () — перемещает загруженный файл в новое место.

Описание

```
move_uploaded_file($from, $to);
```

Функция проверяет, является ли файл **from** загруженным на сервер (переданным по протоколу HTTP POST). Если файл действительно загружен на сервер, он будет перемещён в место, указанное в аргументе **to**.

Параметры

- **from** - путь к загруженному файлу.
- **to** - путь, по которому необходимо переместить файл.

Такая проверка особенно важна в том случае, если существует шанс того, что какие-либо действия, производимые над загруженным файлом, могут открыть его содержимое пользователю или даже другим пользователям системы.

Возвращаемые значения

- В случае успешного выполнения возвращает **true**.
- Если **from** не является загруженным файлом, никаких действий не предпринимается и move_uploaded_file() возвращает **false**.
- Если **from** является загруженным файлом, но не может быть перемещён по каким-либо причинам, никаких действий не предпринимается и move_uploaded_file() возвращает **false**. Кроме того, отображается **предупреждение**.

Код обработчика формы для перемещения файла в директорию постоянного хранения представлен в листинге example_3_2.

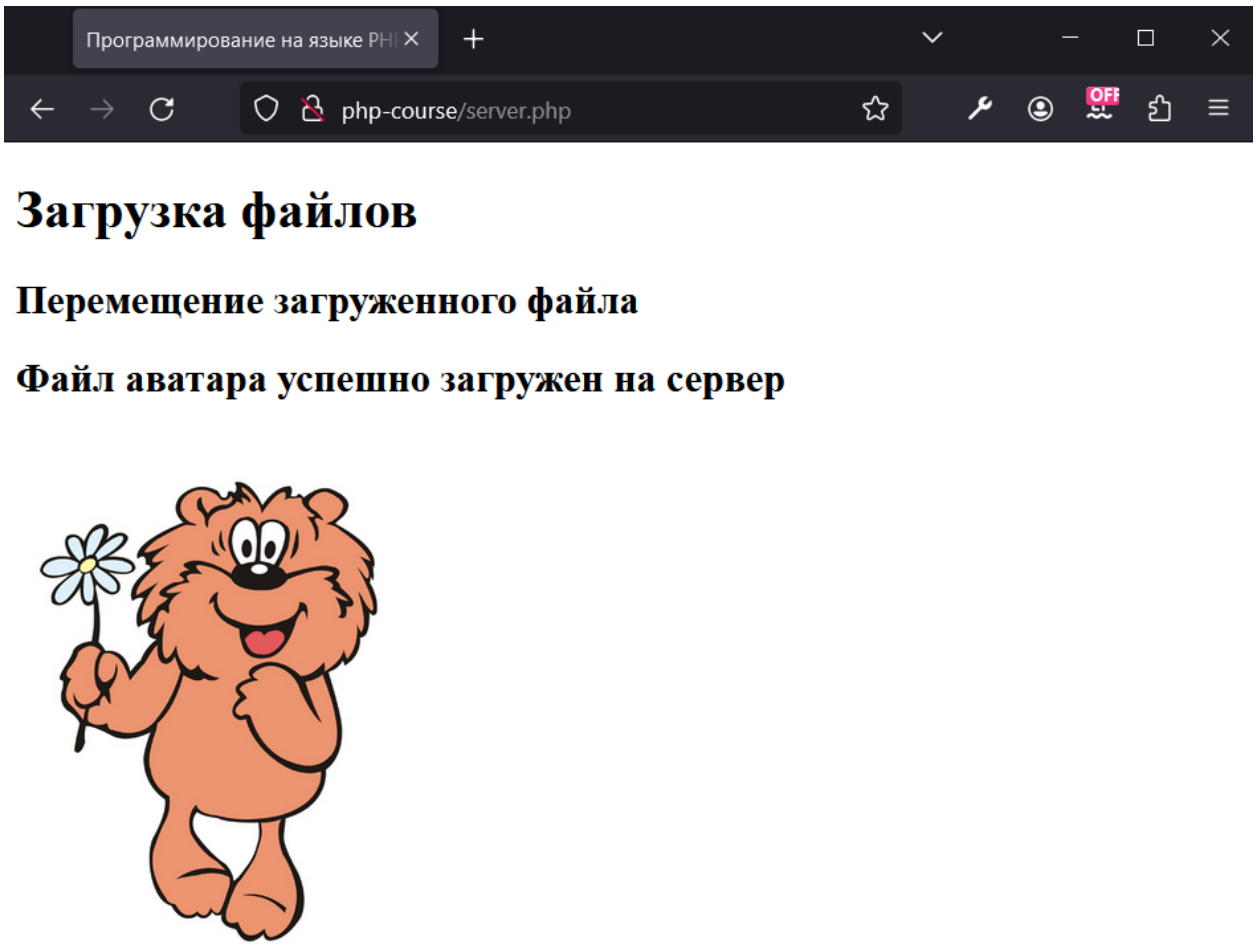
Выполняя тестирование учтите:

- сценарий обработчика все-таки ждет от вас **изображение** аватара;
- сценарий не выполняет никаких проверок на возникновение ошибок (что в реальных приложениях недопустимо).

example_3/server.php. Перемещение загруженного файла

```php
<?php
    // текущее расположение файла
    $current_path = $_FILES['avatar']['tmp_name'];
    // оригинальное имя файла
    $filename = $_FILES['avatar']['name'];
    // место постоянного хранения файла
    $new_path = __DIR__ . '/upload/' . $filename;
    // перемещение загруженного файла
    // если перемещение выполнено успешно, то ...
    if (move_uploaded_file($current_path, $new_path)) {
        // --> выводим сообщение об успешной загрузке файла изображения
        echo "<h2>Файл аватара успешно загружен на сервер</h2>";
        // --> выводим изображение аватарки
        echo "<img src='/upload/" . $filename . "' width='250px'>";
    } else {
        // если во время перемещения возникла ошибка
        echo "<h3>Что-то пошло не так...</h3>";
    };
?>
```

Если загружен не графический файл-изображение – нет гарантии, что ваша аватарка вас порадует...



> ***Важно.*** В данной теме мы выполняем действия по **загрузке** и **сохранению** файлов на сервере. **Проверять** и **анализировать** принятые от пользователя файлы будем потом.

Мультизагрузка

Использование нескольких элементов type = "file"

Загрузку **нескольких файлов** можно реализовать, используя, например, различные значения **name** для тега input:

```
<input type="file" name="file1">
<input type="file" name="file2">
```

```
<input type="file" name="file3">
```

Но гораздо оптимальнее одновременно загружать **несколько файлов** и автоматически получить их **в виде массива** (опять массивы):

```
<input type="file" name="file[]">
<input type="file" name="file[]">
<input type="file" name="file[]">
```

> ***К сведению***. Директива **max_file_uploads** файла **php.ini** позволяет устанавливать максимальное количество файлов, которые могут быть загружены на сервер за один раз. По умолчанию это 20.

Реализуем код множественной загрузки файлов.

example_4/form.html. Форма загрузки нескольких файлов

```
<form method="POST" action="server.php" enctype="multipart/form-data">
    <label>Выберите файлы для загрузки:
    <input type="file" name="myfile[]"><p>
    <input type="file" name="myfile[]"><p>
    <input type="file" name="myfile[]"><p>
    </label><p>
    <input type="submit" name="loader" value="Загрузить файл">
</form>
```

> ***Важно***. Проверьте, что в директории размещения файла сценария у вас имеется каталог **upload**.

Загруженные файлы принимаются как массив:

1. $_FILES['myfile']['name'][0] - имя первого файла

2. $_FILES['myfile']['name'][1] - имя второго файла
3. и т.д.

example_4/server.php. Перемещение нескольких файлов

```php
<?php
    // тестовый вывод
    // echo "<pre>";
    // print_r($_FILES['myfile']);
    // echo "</pre>";
    if(isset($_FILES['myfile'])){
        // количество принятых файлов
        $count = count($_FILES["myfile"]["name"]);
        // запустим цикл по массиву файлов
        for ($i=0; $i < $count; $i++) {
            // текущая временная директория
            $current_path = $_FILES["myfile"]["tmp_name"][$i];
            // оригинальное имя файла
            $filename = $_FILES["myfile"]['name'][$i];
            // директория постоянного хранения
            $new_path = __DIR__ . '/upload/' . $filename;
            // перемещение файла
            move_uploaded_file($current_path, $new_path);
        }
        // если загрузка успешна все файлы должны быть в
        директории хранения
        echo "<h1>Проверьте папку upload</h1>";
    }
?>
```

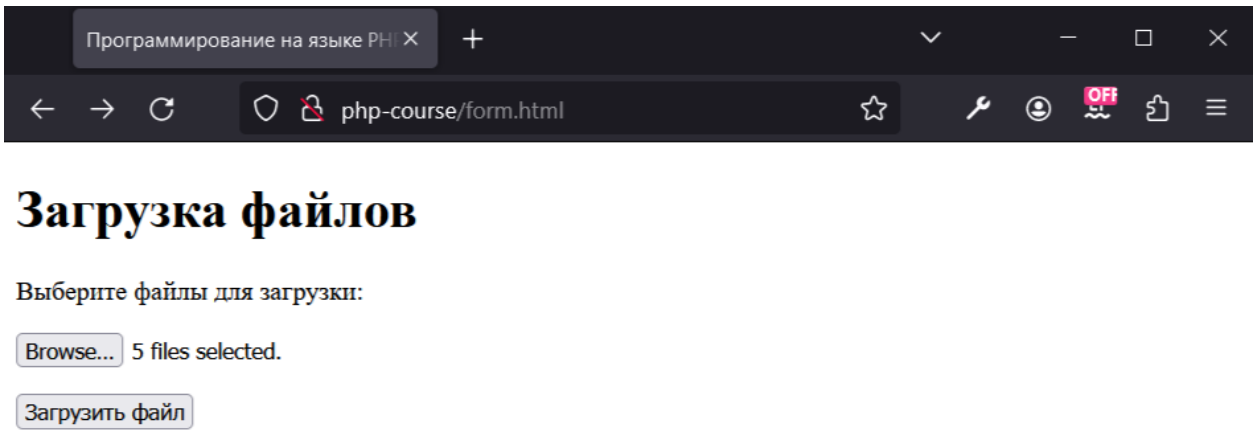
Использование одного элемента type = "file"

Но есть еще один способ выполнить множественную загрузку файлов на сервер. Это делается с использованием **одного элемента input** и атрибута **multiple**.

```
<input type="file" name="myfile[]" multiple>
```

> ***Примечание.*** Множественный выбор осуществляется **удержанием** на клавиатуре клавиши **Ctrl** при выборе файлов.

Форма загрузчика становится более универсальной, количество загружаемых файлов не ограничено (с точки зрения интерфейса программы, но не возможностей сервера), при этом сценарий-обработчик **массива загружаемых файлов** остается неизменным.



example_5/form.html. Загрузка нескольких файлов через свойство multiple

```
<form method="POST" action="server.php" enctype="multipart/form-data">
    Выберите файлы для загрузки: <p>
    <input type="file" name="myfile[]" multiple><p>
    <input type="submit" name="loader" value="Загрузить файл">
</form>
```

Код обработчика формы (**server.php**) полностью идентичен коду обработчика **example_4/server.php**.

Выбор имени для файла

До сих пор мы сохраняли файл на сервере под его оригинальным именем. В реальных проектах такое сохранение приведет к **перезаписи** файлов с одинаковыми именами. Выбор имени, под которым файл будет храниться на сервере, это уже задача не пользователя, а разработчика приложения.

Есть несколько подходов к процессу генерации имени, но общий смысл – **получение уникального имени** для каждого сохраняемого файла.

> ***Примечание***. Получение 100% уникального имени достаточно сложная задача. Поэтому речь скорее может идти о получении имени, приближенного к уникальному.
>
> Подробнее познакомиться с вопросом можно в открытых источниках.

Генерация уникального имени

Первая функция, которую рассмотрим uniqid.

uniqid

uniqid () — генерирует уникальный ID.

Описание

```
uniqid($prefix, $more_entropy);
```

Получает **уникальный идентификатор** с префиксом, основанный на текущем времени в микросекундах.

Параметры

- **prefix** - полезно, если идентификаторы генерируются одновременно на нескольких хостах и генерация идентификаторов производится в одну и ту же микросекунду. По умолчанию **prefix=""**. С пустым параметром prefix, возвращаемая строка будет длиной в 13 символов.
- **more_entropy** - если равен **true**, то функция uniqid() добавит дополнительную энтропию, возвращаемая строка будет длиной в 23 символа. По умолчанию имеет значение **false**.

Возвращаемые значения

Возвращает уникальный идентификатор в виде строки.

Тестируем функцию в следующем примере.

example_6. Тестируем функцию uniqid

```
<?php
    // вызов функции с параметрами по умолчанию
    $filename1 = uniqid();
    echo $filename1, "<p>";
    // вызов функции с prefix = "host_"
    $filename2 = uniqid("host_");
    echo $filename2, "<p>";
    // вызов функции с дополнительной энтропией
    $filename3 = uniqid('', true);
```

```
    echo $filename3, "<p>";
```

Функция не гарантирует получения уникального значения, возможна ситуация, когда будет возвращен неуникальный идентификатор для процесса. Для увеличения вероятности получения уникального значения необходимо использовать параметр **more_entropy**.

Вместо uniqid можно использовать функции, возвращающие текущую метку времени Unix:

- **time** – возвращает количество секунд, прошедших с начала эпохи Unix;
- **microtime** – возвращает количество секунд + микросекунд, прошедших с начала эпохи Unix.

> ***Примечание***. Эпоха Unix отсчитывается с 1 января 1970 0:00:00 GMT.

time

time () — возвращает текущую метку системного времени Unix.

Описание

```
time();
```

Возвращает количество секунд, прошедших с начала эпохи Unix до текущего времени.

Параметры

У функции нет параметров.

Возвращаемые значения

Возвращает текущую метку системного времени.

Пример:

```
<?php
    $filename = time();
    echo $filename;
```

microtime

microtime () — возвращает текущую метку времени Unix с микросекундами.

Описание

```
microtime($float);
```

Функция возвращает текущую метку времени Unix с микросекундами.

Параметры

- **float** - если указано и установлено в **true**, microtime() возвратит число с плавающей точкой вместо строки, как описано в разделе возвращаемых значений ниже. По умолчанию установлено в **false**.

Возвращаемые значения

По умолчанию microtime() возвращает **строку** в формате "**msec sec**", где:

- **sec** — количество секунд с начала эпохи Unix;
- **msec** — это количество микросекунд, прошедших после sec.

Если параметр **float** установлен в **true**, то microtime() возвратит результат в **вещественном виде** (float), представляющий собой текущее время в секундах, прошедших с начала эпохи Unix с точностью до **микросекунд**.

example_7. Тестируем функцию microtime

```
<?php
```

```
    // вызов функции с параметрами по умолчанию
    $filename1 = microtime();
    echo $filename1, "<p>";
    // 0.70888000 1658316606
    // -> 1658316606 - sec
    // -> 0.70888000 - msec
    // вызов функции с float = true
    $filename2 = microtime(true);
    echo $filename2, "<p>";
    // 1658316606.7089
?>
```

Усложним пример, придадим имени больше уникальности с помощью функции **md5**(). Чтобы понять, что такое **md5** сначала несколько слов о понятии хеширования.

> ***Примечание.*** **Хеширование** — это процесс преобразования любого массива данных в выходную строку фиксированной длины. Это преобразование проводится с помощью, так называемых, **хеш-функций**.

Такая функция на входе получает массив данных, а на выходе возвращает так называемый хеш (хеш-суму) – **уникальную строку фиксированной длины**.

md5 или Message Digest 5 это 128-битный алгоритм хеширования который разработан профессором Рональдом Ривестом. Представляется как последовательность из 32 шестнадцатеричных цифр, к примеру: 1bc29b36f623ba82aaf6724fd3b16718.

> ***Примечание.*** Понятие "128-битный" относится к размеру секретного

ключа, известного только вам и людям, с которыми вы хотите обмениваться информацией.

md5

md5() — возвращает md5-хеш строки.

Описание

```
md5($string, $binary);
```

Вычисляет md5-хеш строки string, используя алгоритм MD5 RSA Data Security, Inc. и возвращает этот хеш.

Параметры

- **string** - строка.
- **binary** - если необязательный аргумент binary имеет значение **true**, то возвращается бинарная строка из 16 символов. По умолчанию имеет значение **false**.

Возвращаемые значения

Возвращает хеш в виде 32-символьного шестнадцатеричного числа.

example_8. Тестируем функцию md5

```
<?php
    // uniqid + md5
    printf("
        uniqid()=%s; md5(uniqid())=%s <p>
        ",
        $filename1 = uniqid(),
        md5($filename1)
    );
```

```
    // time + md5
    printf("
        time()=%s; md5(time())=%s <p>
        ",
        $filename2 = time(),
        md5($filename2)
    );
    // microtime + md5
    printf("
        microtime()=%s; md5(microtime())=%s <p>
        ",
        $filename2 = microtime(),
        md5($filename2)
    );
?>
```

Таким образом, я показал, как можно получить уникальное имя для хранимого на сервере файла. Но полное имя состоит из имени и расширения. Теперь посмотрим, как можно получить расширение из полного имени загружаемого файла.

Как обычно, нет одного хорошего способа, есть несколько разных способов, выбрать из которых – задача разработчика.

Получение расширения загружаемого файла

Стандартное имя файла состоит из **двух частей**, разделенных точкой:

- собственно **имя** файла;
- **расширение**, определяющее тип файла.

Собственно имя файлу дает пользователь, а тип файла обычно задается программой автоматически при его создании.

И хорошо, если полное имя файла, например, такое – **myfile.txt**.

Используя функцию **explode**() разбиваем строку в массив, тогда первый элемент массива – имя файла; второй элемент массива – его расширение.

```
$arr = explode('.', 'myfile.txt');
echo 'Имя файла: ', $arr[0], '<p>';  // myfile
echo 'Расширение файла: ', $arr[1];  // txt
```

Все было бы отлично, если бы нам удалось убедить пользователя называть свои файлы правильно. Но пользователь вправе назвать файл, например, так – **my.file.txt**.

Поэтому модернизируем первый способ выделения **расширения** из **полного** имени файла и получим следующий код.

example_9/form.html. Форма загрузки

```
<form method="post" action="server.php" enctype="multipart/form-data">
    Выбор файла для загрузки: <input type="file" name="myfile"><p>
    <input type="submit" name="load" value="Загрузить файл">
</form>
```

example_9/server.php. Получаем расширение файла

```
<?php
    // разобъем строку в массив
    $arr = explode('.',$_FILES['myfile']['name']);
```

```
    // установим указатель на последний элемент массива
    $ext = end($arr);
    // выводим расширение
    echo "Расширение файла: " . $ext;
?>
```

Рассмотрим еще один способ, на этот раз на базе **стандартной** PHP функции **pathinfo**().

pathinfo

pathinfo() — возвращает информацию о пути к файлу.

Описание

```
pathinfo ($path, $flags);
```

Возвращает информацию о **path** в виде **ассоциативного массива** или **строки**, в зависимости от **flags**.

Параметры

- **path** - анализируемый путь.
- **flags** - если указан, то задаёт, какой из элементов пути будет возвращён:
  - PATHINFO_DIRNAME
  - PATHINFO_BASENAME
  - PATHINFO_EXTENSION
  - PATHINFO_FILENAME.

  Если **flags** не указан, то возвращаются **все доступные элементы**.

Возвращаемые значения

Если параметр **flags** не передан, то возвращаемый **ассоциативный массив** будет содержать следующие элементы: **dirname**, **basename**, **extension** (если есть) и **filename**.

Протестируем работу функции на реальном примере.

example_10/form.html. Форма загрузки файла

```
<form method="post" action="server.php" enctype="multipart/form-data">
    Выбор файла для загрузки: <input type="file" name="myfile"><p>
    <input type="submit" name="load" value="Загрузить файл">
</form>
```

example_10/server.php. Тестируем функцию pathinfo

```
<?php
    if(isset($_FILES['myfile'])){
        // $_FILES['myfile']['name'] - имя загружаемого файла
        $fileInfo = pathinfo($_FILES['myfile']['name']);
        // выводим ассоциативный массив
        // содержащий информацию о пути к файлу
        echo "<pre>";
        var_dump($fileInfo);
        echo "</pre>";
        // из массива получаем расширение файла
        echo 'Расширение файла: <b>' . $fileInfo['extension'] . '</b>';
    }
?>
```

Протестируем функцию **pathinfo**() с флагом **PATHINFO_EXTENSION**.

example_11/form.html. Форма загрузки файла

```
<form method="post" action="server.php" enctype="multipart/form-data">
    Выбор файла для загрузки: <input type="file" name="myfile"><p>
    <input type="submit" name="load" value="Загрузить файл">
</form>
```

example_11/server.php. Функция pathinfo с флагом PATHINFO_EXTENSION

```
<?php
    if(isset($_FILES['myfile'])){
        // вызовем функцию pathinfo с флагом PATHINFO_EXTENSION
        $extension = pathinfo($_FILES['myfile']['name'], PATHINFO_EXTENSION);
        // выводим расширение файла
        echo 'Расширение файла: <b>' . $extension . '</b>';
    }
?>
```